САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет программной инженерии и компьютерной техники (ПИиКТ)
Дисциплина: «Расширенный курс по бэкенд-разработке в среде Node.JS»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
«Реализация REST API на основе boilerplate»

Выполнил:
студент гр. К3340 (поток БР1.1)
Галилей Кирилл Дмитриевич

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

1. ВАРИАНТ

Вариант 1. «Приложение для бронирования столиков в ресторанах»

Ключевая функциональность системы согласно заданию:
- Вход (аутентификация с выпуском JWT-токенов доступа)
- Регистрация (создание аккаунтов клиентов и менеджеров с хэшированием паролей)
- Личный кабинет пользователя (профиль, контактные данные, история)
- Поиск ресторанов с фильтрацией по кухне, расположению и цене
- Страница с информацией о ресторане (меню, фото, отзывы, столики)
- История бронирований пользователя (создание, просмотр, отмена)

2. ЗАДАЧА

1) Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд (можно взять и express-typeorm-boilerplate)
2) С учётом имеющихся данных по результатам Д31 и Д32 реализуйте:
    - модели
    - представления
    - контроллеры
3) В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

3. ХОД РАБОТЫ

3.1. Архитектура приложения и стек технологий

Стек: Node.js 20, TypeScript 5, Express.js 4, TypeORM 0.3+, PostgreSQL 16.
Структура проекта:
- src/config/ - настройки окружения и подключение TypeORM DataSource
- src/models/ - сущности User, Restaurant, Table, Reservation, MenuItem, Review
- src/controllers/ - обработка запросов и бизнес-логика (Auth, Restaurant, Table, Reservation, Review, Menu)
- src/middlewares/ - аутентификация JWT и проверка прав доступа
- src/swagger.ts - интерактивная документация /api/docs
- src/app.ts - точка входа приложения Express

3.2. Модели TypeORM

Реализовано 6 сущностей со строгой типизацией и связями:
- User: id, email, password, firstName, lastName, phone, role
- Restaurant: id, name, description, cuisine, city, address, priceRange, averageRating
- Table: id, tableNumber, capacity, location, isActive, restaurant_id
- Reservation: id, reservationDate, timeSlot, guestsCount, status, user_id, restaurant_id, table_id
- MenuItem: id, name, description, category, price, isAvailable, restaurant_id
- Review: id, rating (1-5), comment, user_id, restaurant_id

3.3. Бизнес-логика контроллеров

- AuthController: хеширование bcrypt, генерация JWT токенов
- RestaurantController: фильтрация по кухне, городу и ценовому диапазону
- TableController: добавление столиков с контролем уникальности номера

- ReservationController: проверка доступности столика на временной слот (защита от овербукинга, возврат 409 Conflict)
- ReviewController: публикация отзыва и автоматический пересчет рейтинга ресторана

3.4. Запуск и локальное окружение
Запуск PostgreSQL через docker-compose.yml (postgres:16-alpine).
Сборка: npm run build, запуск: npm start.
Исходный код упакован в source_code_lab1.zip.

4. ВЫВОД

В ходе выполнения лабораторной работы №1 на основе расширенного boilerplate реализовано полнофункциональное серверное приложение REST API на TypeScript, Express и TypeORM с СУБД PostgreSQL. Реализованы все модели, контроллеры и middleware, предусмотрена защита от овербукинга и авторизация по ролям. Сервер протестирован, документирован в Swagger UI и упакован в архив исходного кода.